

Lab Report 5: I2C and SPI Drivers with MPU6050 and HD44780 LCD Integration

Authors:

J.M. Páez Sandoval, 37945

L.A. Lugo Muñiz, 37915

D. Lopez Moreno, 37984

C.A. Martínez Macías, 33369

Teacher : Carlos A. Villareal

CETYS Universidad

Facultad de Ingeniería

Curso: Microcontroladores y Sistemas Embebidos

Fecha: May 29, 2026

Abstract—This report documents the bare-metal implementation of I2C and SPI communication drivers for the NUCLEO-F411RE development board (STM32F411RE, ARM Cortex-M4). The I2C1 driver operates at 100 kHz standard mode on PB8 (SCL) and PB9 (SDA) and is used to interface with the MPU6050 inertial measurement unit (IMU), reading 3-axis accelerometer and gyroscope data via a 14-byte burst read. The SPI1 driver operates in Mode 3 at approximately 3.125 MHz on PA5/PA6/PA7, with software-controlled chip select on PA9. Application output is rendered on a 16×2 HD44780 LCD in 4-bit parallel mode, displaying real-time gyroscope X, Y, Z values. Serial telemetry is simultaneously transmitted over USART2 at Index Terms—STM32F411RE, I2C1, SPI1, MPU6050, HD44780, LCD, bare-metal, CMSIS, gyroscope, accelerometer, NUCLEO-F411RE.

I. INTRODUCTION

Serial communication protocols are fundamental to embedded systems design. The I2C (Inter-Integrated Circuit) and SPI (Serial Peripheral Interface) buses allow a microcontroller to communicate with sensors and peripherals using a small number of shared signal lines. I2C uses two wires (SCL and SDA) with an address-based scheme, enabling multiple devices on the same bus. SPI uses four wires (SCK, MISO, MOSI, CS) and achieves higher data rates through a full-duplex, chip-select-based topology.

This laboratory practice implements both protocols on the STM32F411RE using a bare-metal C architecture that follows the layered driver philosophy established in previous labs. The I2C driver interfaces with the MPU6050 IMU to read gyroscope data, while the SPI driver provides a general-purpose communication channel. Application-level logic ties both subsystems together:

gyroscope readings are formatted and displayed on a 16×2 HD44780 LCD in 4-bit parallel mode and simultaneously streamed over USART2 for serial monitoring.

The practice reinforces modular hardware abstraction: each driver encapsulates a single peripheral, exposing a clean API to the layer above without exposing register-level details to the application.

II. MATERIALS AND EQUIPMENT

The following hardware and software resources were used:

- NUCLEO-F411RE board (STM32F411RE, Cortex-M4)
- MPU6050 IMU module (I2C, 0x68)
- 16×2 HD44780 LCD (4-bit parallel, PB5/PB6/PB7/PB10/PB12/PB13)
- Breadboard, jumper wires, USB-A to Mini-USB cable
- Personal computer, Windows 11
- Visual Studio Code with Serial Monitor extension
- MSYS2 / MinGW64, arm-none-eabi-gcc cross-compiler
- OpenOCD, custom Makefile, CMSIS STM32F411xE headers

III. DEVELOPMENT

A. Project Structure and Layered Architecture

The firmware follows a strict layered architecture. At the lowest level, `gpio_driver` manages all GPIO register operations, including

alternate-function configuration for I2C and SPI pins. The `i2c_driver` abstracts I2C1 peripheral registers (CR1, CR2, CCR, TRISE, SR1, SR2, DR), exposing a four-function API for register-addressed and raw read/write operations. The `spi_driver` abstracts SPI1 (CR1, CR2, SR, DR), providing transmit, receive, and chip-select functions. Above these, the `mpu6050` module implements sensor initialization and a 14-byte burst read that extracts accelerometer and gyroscope raw values. The `LCD_driver` implements the HD44780 power-on sequence and 4-bit parallel interface. The serial module wraps `uart_driver` for formatted telemetry output.

B. I2C Driver

The `i2c_driver` initializes I2C1 at 100 kHz standard mode. The initialization sequence performs a software reset via CR1.SWRST, sets the peripheral frequency in CR2 to 16 (matching the 16 MHz HSI clock), computes $CCR = fPCLK / (2 \times fSCL) = 16,000,000 / 200,000 = 80$, and $TRISE = (fPCLK_MHz \times t_rise_max_ns / 1000) + 1 = (16 \times 1000 / 1000) + 1 = 17$. PB8 and PB9 are configured as AF4 (I2C1), open-drain output with pull-up resistors enabled.

The `i2c_writeRegDevice()` and `i2c_readRegDevice()` functions implement the standard I2C register-write and register-read sequences with repeated-start. A timeout counter macro (I2C_WAIT) guards each status-bit poll, issuing a STOP condition on expiry to release the bus and prevent deadlock.

C. SPI Driver

The `spi_driver` configures SPI1 in Master mode, Mode 3 (CPOL=1, CPHA=1), 8-bit MSB-first frame format, with a baud-rate prescaler of /32 yielding approximately 3.125 MHz on APB2. Software NSS management (SSM=1, SSI=1) is used; chip select is controlled manually through PA9 as a GPIO output. PA5, PA6, and PA7 are configured as AF5 (SPI1) for SCK, MISO, and MOSI respectively.

`spi_transmit()` polls SR.TXE before writing each byte to DR, then waits for SR.BSY to clear and discards the simultaneously received byte to prevent Overrun (OVR) errors. `spi_receive()` drives the clock by writing a dummy byte (0x00) and reads DR after RXNE is set.

D. MPU6050 Module

The `mpu6050_init()` function configures PB8/PB9 as AF4 open-drain with pull-ups, calls `i2c_init()`, and wakes the sensor by writing 0x00 to the PWR_MGMT_1 register (0x6B), disabling sleep mode. The sensor I2C address is 0x68 (AD0 pin tied low)

`mpu6050_readData()` performs a 14-byte burst read starting from register ACCEL_XOUT_H

(0x3B). The MPU6050 stores each axis value as a signed 16-bit big-endian integer across two consecutive registers. The function reconstructs each value by left-shifting the high byte and OR-ing in the low byte: $gx = (buf[8] \ll 8) | buf[9]$, and similarly for gy, gz, ax, ay, az..

E. LCD Driver

The HD44780 LCD is driven in 4-bit parallel mode using six GPIO pins on GPIOB: RS (PB5), E (PB6), D4 (PB7), D5 (PB12), D6 (PB13), D7 (PB10). The initialization sequence follows the HD44780 datasheet power-on procedure: wait >40 ms, send 0x3 three times with defined delays, switch to 4-bit mode (0x2), then configure display (2 lines, 5×8 font), clear, and set entry mode (auto-increment, no shift).

Each byte is sent as two 4-bit nibbles, high nibble first. The enable pulse is generated by asserting E high for at least 1 μs then pulling it low. The application calls `lcd_set_cursor()` and `lcd_print_string()` to update both rows every 500 ms with freshly read gyroscope values.

F. Application Integration

The `main()` function initializes all modules in dependency order: timer → serial → mpu6050 (which calls `i2c_init` internally) → lcd. In the main loop, `mpu6050_readData()` fills an `MPU6050_t` structure with the six raw axis values. The gyroscope X, Y, Z values are formatted into two strings using `utils_snprintf()` and written to LCD row 0 and row 1 respectively. Simultaneously, `serial_printf()` transmits all three values over USART2 at 115200 baud. A 500 ms blocking delay via `timer_delay_ms()` paces the loop.

IV. RESULTS

A. LCD Output Verification

After flashing the firmware, the LCD immediately began displaying gyroscope readings. Figure 1 shows the hardware setup with the LCD powered and displaying a representative set of values: X:-261, Y:177, Z:164. The display updated every 500 ms as the board was manipulated, with all three axes responding correctly to orientation changes, confirming successful I2C communication with the MPU6050 and correct 4-bit parallel LCD operation.



Fig. 1. HD44780 LCD output showing live MPU6050 gyroscope telemetry.

B. Serial Monitor Output

The serial monitor showed continuous telemetry lines in the format `GX:%d GY:%d GZ:%d` at the expected 500 ms interval, matching the LCD values and confirming that both output channels (LCD and UART) operated concurrently without interference.

C. I2C Timing Verification

The CCR value of 80 yields an SCL period of $2 \times 80 \times (1/16 \text{ MHz}) = 10 \mu\text{s}$, corresponding to a clock frequency of 100 kHz. The TRISE value of 17 accommodates the 1000 ns maximum rise time specified in the I2C standard for 100 kHz operation. No NACK or timeout events were observed during extended testing.

D. Functional Compliance Summary

F R	Require ment	Implementation	Stat us
F R 1	Enable I2C1 APB1 clock	RCC->APB1ENR = I2C1EN	✓
F R 2	Configure CCR for 100 kHz	CCR = 80	✓
F R 3	Generate START condition	CR1 = I2C_CR1_START	✓

F R 4	Read 14-byte burst from MPU6050	i2c_readRegDevice(0x68, 0x3B, buf, 14)	✓
F R 5	SPI1 Master Mode 3 @ ~3 MHz	CR1: MSTR SSM SSI CPOL CPHA BR/32	✓
F R 6	Software CS control	spi_cs_enable/disable via PA9	✓
F R 7	Display gyro data on LCD	lcd_print_string on rows 0 and 1	✓

TABLE I — Functional Requirements Compliance

V. CONCLUSION

This laboratory practice successfully implemented bare-metal I2C and SPI communication drivers on the STM32F411RE and demonstrated their integration in a complete sensor-display telemetry pipeline. The I2C1 driver, configured at 100 kHz with correctly computed CCR and TRISE values, communicated reliably with the MPU6050 IMU across all tested orientations. The SPI1 driver, configured in Mode 3 at 3.125 MHz, provides a reusable general-purpose channel for future peripherals.

The layered architecture — `gpio_driver` → `i2c_driver` / `spi_driver` → `mpu6050` → `LCD_driver` / `serial` — isolated hardware concerns from application logic and allowed both output channels to operate concurrently without register-level conflicts. The MPU6050 14-byte burst read demonstrated an efficient I2C usage pattern that minimizes bus overhead by retrieving all sensor axes in a single transaction.

Future work could enable the accelerometer data (also captured in the `MPU6050_t` struct) alongside the gyroscope, implement interrupt-driven I2C to eliminate blocking waits, or add DMA-based SPI transfers for higher-bandwidth peripherals.

REFERENCES

- [1] STMicroelectronics, "STM32F411xC/E Reference Manual (RM0383)," Rev. 3, 2019.
- [2] STMicroelectronics, "NUCLEO-F411RE User Manual (UM1724)," Rev. 14, 2021.
- [3] InvenSense, "MPU-6000/MPU-6050 Product Specification," Rev. 3.4, 2013.
- [4] Hitachi Semiconductor, "HD44780U Dot Matrix Liquid Crystal Display Controller/Driver," Rev. 0, 1998.
- [5] ARM Limited, "Cortex-M4 Devices Generic User Guide (DUI0553)," 2010.